

SIMATS ENGINEERING
CO2 - ASSESSMENT TOOL 2
Scenario-Based Questions

COURSE NAME: Deep Learning COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO 2: Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches. (BTL-4)

CO Weightage: 15% Assessment Tool Weightage: 40%

Duration: 90 minutes Marks: 25

Code of Conduct:

I **G Naveen Kumar Reddy** (Name / Reg No: 192525288) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G Naveen Kumar Reddy

Scenario-Based Questions

CO-AT2

QUESTIONS: 05 Total Marks: 25

1. Unsupervised Training of Neural Networks, Auto Encoders (5 Marks)

Scenario: A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

Answer:

Recommended technique: An Autoencoder-based unsupervised anomaly detection model (optionally a deep/stacked autoencoder, or an LSTM-Autoencoder since ECG data is sequential/time-series in nature).

Justification:

- The ECG records are unlabeled, so supervised classification cannot be used directly — there are no "normal/abnormal" tags to train against.
- Autoencoders learn a compressed latent representation of the input by training the network to reconstruct its own input (encoder → bottleneck → decoder), which is exactly the kind of representation learning needed here.
- Since the autoencoder is trained mostly (or only) on normal ECG signals, it learns to reconstruct normal heartbeat patterns very well.
- When an abnormal (previously unseen) ECG pattern is passed through the trained autoencoder, the reconstruction error (difference between input and output) will be significantly higher, because the network has not learned to represent that pattern efficiently.

Workflow:

1. Data collection & preprocessing: Segment continuous ECG signals into fixed-length beats/windows; normalize amplitude and remove noise.
2. Model design: Build an encoder (series of dense or convolutional/LSTM layers) that compresses the ECG window into a low-dimensional latent vector, and a decoder that reconstructs the original signal from this latent vector.
3. Training: Train the autoencoder using unsupervised reconstruction loss (e.g., Mean Squared Error) predominantly on normal ECG data.
4. Threshold setting: Compute reconstruction error on a validation set of normal signals and set a statistical threshold (e.g., mean + $k \times$ standard deviation).
5. Anomaly detection: For new/unseen ECG windows, compute reconstruction error; if the error exceeds the threshold, flag the record as an abnormal pattern for clinical review.
6. Evaluation: Validate against any available clinician-labeled samples to tune the threshold and reduce false positives/negatives.

2. Auto Encoders, Representation Learning (5 Marks)

Scenario: A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

Answer:

An autoencoder is a neural network trained to copy its input to its output through a narrow "bottleneck" hidden layer, and it is naturally suited for dimensionality reduction because the bottleneck forces the network to learn only the most important, compressed features of the data.

How it solves the problem:

- **Encoder:** The high-dimensional customer purchase data (e.g., purchase frequency, category-wise spend, recency, basket size, brand preference, etc.) is fed into the encoder, which progressively compresses it through smaller hidden layers down to a low-dimensional latent/bottleneck representation.
- **Bottleneck (latent space):** This compressed vector captures the essential underlying patterns of a customer's purchasing behaviour in far fewer dimensions than the original feature set, while discarding redundant or noisy information.
- **Decoder:** The decoder attempts to reconstruct the original purchase data from this compressed representation; the network is trained by minimizing the reconstruction error, forcing the latent space to retain the most meaningful structure of the data.
- **Using the latent features:** Once trained, only the encoder is used — the low-dimensional latent vectors (instead of the raw high-dimensional features) are extracted for every customer.
- **Segmentation:** These compact latent representations are then fed into a clustering algorithm (e.g., K-Means, hierarchical clustering, or DBSCAN) to group customers into meaningful segments (e.g., bargain hunters, brand-loyal high spenders, occasional buyers).

Advantage over traditional methods (e.g., PCA): Autoencoders can capture non-linear relationships between purchase features, which linear techniques like PCA cannot, generally leading to more meaningful and well-separated customer segments.

3. Width and Depth of Neural Networks, Activation Functions (5 Marks)

Scenario: A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

Answer:

Problem diagnosis: A shallow network has too few hidden layers to learn the hierarchical, complex features (edges → facial parts → identity-level features) needed for facial recognition, so it underfits the task.

Recommended architectural changes:

- Increase depth: Move from a shallow network to a deep convolutional neural network (CNN) with multiple stacked convolutional and pooling layers. Depth allows the network to build a hierarchy of features — early layers learn edges/textures, middle layers learn facial components (eyes, nose, mouth), and deeper layers learn identity-discriminative, high-level features.
- Balance depth and width: Along with adding depth, widen key layers (more filters/neurons) so the network has enough capacity to represent the wide variety of faces, without making the network so deep/wide that it overfits or becomes hard to train.
- Use skip/residual connections: If depth is increased substantially, incorporate residual (ResNet-style) connections to prevent the vanishing gradient problem and allow effective training of very deep networks.
- Replace saturating activation functions: If the shallow network used Sigmoid or Tanh, replace them with ReLU (or its variants like Leaky ReLU / ELU) in hidden layers, since ReLU avoids vanishing gradients, speeds up convergence, and works well in deep architectures.
- Use Softmax at the output layer for multi-class identity classification, converting the final layer outputs into class probabilities.
- Add regularization: With increased depth/width, add Batch Normalization and Dropout to stabilize training and prevent overfitting.

Result: These changes give the network sufficient depth and representational capacity, combined with non-saturating activations, allowing it to learn discriminative, hierarchical facial features and significantly improving recognition accuracy.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks (5 Marks)

Scenario: A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

Answer:

A Restricted Boltzmann Machine (RBM) is a generative, unsupervised, two-layer neural network (visible layer + hidden layer, with no intra-layer connections) that learns the probability distribution underlying its input data — making it well suited for this labeled-data-free recommendation task.

How RBMs are applied:

- Data representation: Each visible unit represents a movie, and the user's viewing/rating history is encoded as the input vector (e.g., watched/not watched, or rating value) fed to the visible layer.
- Unsupervised training: The RBM is trained using Contrastive Divergence, adjusting weights so that the hidden layer learns latent taste factors (hidden units) that explain the patterns of movies watched together, without needing any explicit labels — the model simply learns to reconstruct the observed viewing pattern.

- Learning latent preferences: The hidden units end up capturing abstract features such as genre preference, actor preference, or viewing mood, which are not explicitly provided but emerge purely from the co-occurrence patterns in the data.
- Generating recommendations: For a given user, their known viewing history activates the hidden units (forward pass); the network then reconstructs the visible layer (backward pass), producing probabilities for movies the user hasn't watched yet — movies with high reconstructed probability become the recommendations.
- Scalability: RBMs can be trained efficiently on large, sparse user-movie matrices, and multiple RBMs can be stacked into a Deep Belief Network for even richer representation learning if needed.

This approach allows the streaming service to build a collaborative-filtering-style recommender purely from unlabeled viewing history.

5. Deep Learning Applications, Representation Learning (5 Marks)

Scenario: A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

Answer:

Problem framing: This is a time-series anomaly/defect detection problem using continuous, high-frequency sensor data (temperature, vibration, pressure, current, etc.), where defective instances are typically rare compared to normal operation.

Proposed solution:

- Data preprocessing: Collect multivariate sensor streams and segment them into fixed-size time windows; normalize/scale each sensor channel and handle missing values.
- Representation learning stage: Use an LSTM/GRU-based Autoencoder (or a 1D-CNN Autoencoder) to learn a compressed latent representation of "normal" operating patterns directly from raw sensor sequences — this removes the need for manual feature engineering and captures temporal dependencies between successive sensor readings.
- Training: Train the autoencoder mainly on sensor data from known-good production runs, using reconstruction loss (MSE) as the unsupervised training objective.
- Defect detection: For new incoming sensor windows, feed them through the trained encoder-decoder; a high reconstruction error indicates the sensor pattern deviates from normal behaviour, flagging a likely defective product/process.
- Refinement with supervision (if some labels exist): If a small set of labeled defective/normal samples is available, the learned latent representations can be fed into a lightweight supervised classifier (e.g., a dense neural network or SVM) for improved precision — combining representation learning with fine-tuned classification.
- Real-time deployment: Since readings arrive every second, the model should run inference in a streaming/sliding-window fashion, raising alerts as soon as anomaly scores cross the threshold, enabling immediate corrective action on the production line.

Benefit: This design leverages unsupervised representation learning to handle the scarcity of labeled defect data while still capturing the complex temporal patterns in high-frequency sensor streams, enabling accurate and timely defect detection.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand